

Introducere în programare - curs și laborator - suport electronic

(C) 2020 Bogdan Pătruț

Cele mai bune practici în scrierea codului

În cadrul acestei lecții vom discuta despre câteva practici de scriere a codului în C/C++, pe care le recomandăm studenților (*clean code*, *cod curat*): nume adecvate, scrierea funcțiilor, când folosim și cum folosim comentarii, formatarea textului, testarea codului.

Meniu

[Pagina principala](#) [Cursul 1](#) [Laboratorul 1](#)

1. Ce este codul curat?

Codul curat este codul scris în așa fel încât poate fi citit cu ușurință și îmbunătățit ulterior de către un programator, altul decât autorul original. **De ce este important să scriem cod curat?**

Metafora geamurilor sparte

O clădire care are geamuri sparte arată de parcă nimănui nu îi pasă de ea. Deci nici altor persoane nu le mai pasă și permit ca alte geamuri să fie sparte. Eventual, se vor sparge și ele.

Acest lucru se aplică și în cod.

2. Nume adecvate

Nume există oriunde în cod. Trebuie să îți denumești:

- variabile
- funcții
- argumente
- clase
- fișiere sursă etc.

Pentru că voi creați cele mai multe nume în cadrul unui program, *trebuie să le faceți bine*. Ce urmează sunt reguli simple de a crea nume bune.

1. Folosiți nume care să evidențieze intenția

Alegerea numelor bune durează, dar va salva mult mai multe lucruri pe viitor. Schimbați-le atunci când găsiți altele mai bune. Numele ar trebui să răspundă la întrebările importante:

- de ce există?
- ce fac?
- cum sunt folosite?

Dacă un nume are nevoie de un comentariu, atunci numele nu își dezvăluie de fapt intenția.

▼ Exemplu: în loc de `int d; // timp trecut în zile`, folosiți: `int timpTrecutInZile;`

2. Evitați dezinformarea

Evitați cuvinte ale căror înțeles poate să varieze. `hp`, `aix`, și `sco` pot fi dezinformative, deoarece sunt nume de pe platformele Unix. Nu vă referiți la grupuri de conturi ca fiind `accountList`, decât dacă este într-adevăr o listă. Cuvântul "list" înseamnă ceva specific pentru programatori. În cazul în care containerul care are conturi nu este chiar `List`, se pot trage concluzii false.

Atenție la nume care pot varia foarte puțin. Cât de mult vă ia ca să vedeți diferențele dintre următoarele două nume: `XYZControllerForEfficientHandlingOfStrings` și `XYZControllerForEfficientStorageOfStrings`? Nu schimbați un nume într-un mod arbitrar doar pentru a satisface un compilator sau interpretor. Numirea seriilor de numere (`a1`, `a2`, .. `aN`) este opusul numirii intenționale. Aceste nume nu sunt dezinformative - sunt pur și simplu noninformative.

▼ Exemplu

```
void copyInts(int a1[], char a2[], int n) {
    for (int i = 0; i < n; i++) {
        a2[i] = a1[i];
    }
}

void copyInts(int source[], int destination[], int length) {
    for (int i = 0; i < length; i++) {
        destination[i] = source[i];
    }
}
```

3. Folosiți nume ce pot fi pronunțate

Oamenii sunt buni la cuvinte. O bună parte din creierul nostru este dedicat conceptului de cuvinte. Iar cuvintele sunt pronunțabile. Deci creați-vă nume pronunțabile. Folosiți nume compuse din verbe și substantive decât formate din abrevieri.

4. Folosiți nume care pot fi căutate

Evitați să folosiți nume formate dintr-o singură literă (le puteți folosi doar ca variabile locale în funcții scurte). Lungimea unui nume trebuie să corespundă cu mărimea contextului său.

▼ Exemplu

```
int e = 0;
```

Folosirea variabilei `e` este greu de găsit. Evitați folosirea constantelor numerice. Este mai ușor să căutați folosința lui `MAX_CLASSES_PER_STUDENT`, decât folosința numărului `7`, deoarece `7` poate avea mai înțelesuri diferite în contexte diferite.

Codul sursă original:

```
for (int j=0; j<34; j++) {
    s += (t[j]*4)/5;
}
```

codul sursă nou:

```
int realDaysPerIdealDay = 4;
const int WORK_DAYS_PER_WEEK = 5;
int sum = 0;
for (int j=0; j < NUMBER_OF_TASKS; j++) {
    int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;
    int realTaskWeeks = (realTaskDays / WORK_DAYS_PER_WEEK);
    sum += realTaskWeeks;
}
```

5. Evitați corelarea între numele variabilelor și cuvintele deja cunoscute

Cititorii codului nu trebuie să traducă mental nume în alte nume pe care le cunosc deja. Evitați alegerea numelor de variabile precum: `a`, `b`, `x1`, `y`. Un contor de buclă poate fi denumit `i` sau `j` sau `k` (dar niciodată `1`!) dacă contextul său este minimal și niciun alt nume nu are vreun conflict cu el. Acest lucru

este valabil pentru că numele de o singură literă pentru contoarele de buclă sunt tradiționale. Dar în cele mai multe situații, numele formate dintr-o singură literă nu sunt o alegere bună. Nu folosiți numele `c`, doar pentru că `a` și `b` au fost alese deja.

6. Nume de metode

În POO, metodele ar trebui să conțină verbe, care să descrie acțiunea pe care trebuie să o execute. De exemplu: `postPayment`, `deletePage`, `save`, `create`, `updateCustomer`. Variabilele de acces trebuie denumite după valoarea lor și prefixat cu `get`, `set`.

```
string name = employee.getName();
customer.setName("mike");
if (paycheck.isPosted())...
```

7. Alegeți câte un nume per concept

Alegeți un cuvânt pentru fiecare concept abstract și folosiți-l, în așa fel încât să asigurați o anumită consistență sistemului vostru. Exemplu: `fetch`, `retrieve`, `get`

8. Adăugați un context semnificativ

Amplasați numele alese în context pentru cititorul vostru, prin includerea lor în structuri de date și funcții bine denumite.

▼ Exemplu

```
firstName, lastName;
street, houseNumber;
city, state, zipcode;
```

Formează o adresă? Putem adăuga context folosind prefixe:

```
addrFirstName, addrLastName, addrState, etc.
```

sau, mai bine, folosim o structură de date denumită `Address`.

Concluzie

Ce este dificil în alegerea de nume bune? Cel mai greu lucru în alegerea de nume este folosirea abilităților descriptive și a unui context cultural comun (este mai mult o problemă de educație decât una tehnică, de afacere sau de administrare). Folosiți unele din regulile descrise și vedeți dacă vă va îmbunătăși lizibilitatea codului vostru. Va da roade în scurt timp și va continua să dea roade și pe termen lung.

3. Funcții

Pentru definirea funcțiilor avem mai multe sfaturi pe care le dăm sub forma unor reguli:

1. **Prima regulă:** Funcțiile trebuie să fie mici!

2. **A doua regulă:** Funcțiile trebuie să fie mai mici de atât!

3. **Blocuri și indentare:**

Blocurile din structurile `if-else`, `while` și așa mai departe trebuie să aibă marimea de maxim o linie. Dacă sunt mai lungi, introduceți o nouă funcție! Acest lucru va menține funcția inclusă mică și adaugă valoare documentară, deoarece funcția apelată în bloc poate avea un nume frumos descriptiv.

Funcțiile nu trebuie să fie suficient de mari pentru a conține structuri imbricate. Prin urmare, nivelul de indentare a unei funcții nu trebuie să fie mai mare de unul sau două. Acest lucru face ca funcțiile să fie mai ușor de citit și de înțeles.

4. **Funcțiile trebuie să facă un singur lucru !**(principiul unicei responsabilități)

Cum testăm dacă o funcție face doar un singur lucru?

1. Are un singur răspuns la întrebarea: "Ce face această funcție?"

2. Dacă puteți extrage o altă funcție din ea cu un nume care nu este doar o retratare a implementării sale.

5. Citirea codului de sus în jos: regula de coborâre

Codul trebuie citit ca o narațiune de sus în jos.

6. Utilizarea numelor descriptive

- Nu este nimic greșit în introducerea unui nume lung.
- Un nume descriptiv lung este mai bun decât un nume enigmatic scurt.
- Un nume descriptiv lung este mai bun decât un comentariu descriptiv lung.
- Utilizați o convenție de denumire care permite citirea mai multor cuvinte în numele funcțiilor.

De exemplu, un nume bun este: `calculeazăEmployeeSalaryBonus`. Fiți consecvenți în numele alese! Utilizați aceleași fraze, substantive și verbe în numele funcțiilor pe care le alegeți pentru modulele voastre.

Numărul de argumente al funcțiilor

Pe baza numărului de argumente, o funcție poate fi:

1. Niladică (0-ară) - niciun argument (număr zero de argumente, caz ideal);
2. Monadică (unare), diadică (binare) - ar trebui evitat
3. Poliadică - nu trebuie utilizat

Motivele comune pentru a trece la un singur argument într-o funcție:

- Este posibil să puneți o întrebare cu privire la acest argument, de exemplu: `boolean fileExists ("MyFile")`
- Este posibil să operați pe acel argument, transformându-l în altceva și returnându-l: `InputStream fileOpen ("MyFile")`
- Alt caz pentru forme monadice este tratarea evenimentelor: În această formă există un argument de intrare, dar nici un argument de ieșire. Programul general este menit să interpreteze apelul funcțional ca un eveniment și să utilizeze argumentul pentru a modifica starea sistemului. Exemplu: `void passwordAttemptFailedNtimes (int try)`

Evitați să utilizați argumentele de tip fanion (*Flag Arguments*). Trecerea unui `boolean` într-o funcție este o practică cu adevărat groaznică, deoarece încălcă principiul responsabilității unice pentru funcție (funcția face un lucru dacă steagul (fanionul) este adevărat și altul dacă steagul este fals). Se complică imediat semnătura funcției.

Funcțiile dyadice sunt mai greu de urmărit și înțeles decât funcțiile monadice.

Iată un exemplu:

```
writeField (nume)
writeField (output-Stream, name)
```

Există, desigur, cazuri în care două argumente sunt adecvate, de exemplu:

```
Punctul p = Punct nou (0,0);
```

Funcțiile triadice, care au trei argumente, sunt în mod semnificativ mai greu de înțeles decât dyadele. Trebuie să aveți un motiv foarte întemeiat pentru a introduce o funcție care necesită 3 argumente.

Obiecte ca argumente

Când o funcție pare să aibă nevoie de mai mult de două sau trei argumente, este bine ca unele dintre aceste argumente să fie înglobate într-o structură de date proprie. De exemplu, în loc de `Circle makeCircle (double x, double y, double raza);`, putem avea ceva de genul `Circle makeCircle (Punct centru, double raza);`, în care `X` și `Y` fac parte dintr-un concept care merită un nume propriu, astfel încât gruparea lor într-o structură de date separată este o idee bună.

Concluzie

Dacă urmați regulile prezentate anterior, funcțiile voastre vor fi: scurte și nu vor avea nevoie de prea multe

descrieri și explicații, deoarece având un nume bine ales, vor ajuta programatorul mai mult decât un antet de comentarii.

4. Comentarii

Commentariile nu sunt "bune aprioric". Ele sunt, în cel mai bun caz, un rău necesar. Utilizarea corespunzătoare a comentariilor este pentru a compensa nerespectarea regulilor noastre în cod. Utilizarea comentariilor poate fi considerată un eșec. Trebuie să le avem pentru că nu ne putem da seama întotdeauna cum să ne exprimăm fără ele, dar utilizarea lor nu este un motiv de sărbătoare. Așadar, când vă aflați într-o poziție în care trebuie să scrieți un comentariu, gândiți-l și vedeți dacă nu există vreun fel de a transforma numele folosite și de a vă exprima în cod.

Comentariile nu compensează codul scris prost

Una dintre cele mai frecvente motivații pentru scrierea comentariilor este codul prost. În loc să-ți petreci timpul scriind comentariile care explică mizeria pe care ai făcut-o, petreceți timp curățând acea mizerie.

Exprimați-vă prin cod

Există cu siguranță momente în care codul este o modalitate slabă pentru explicații, dar de cele mai multe ori este ușor să te exprimi prin cod.

▼ Exemplu

Ce ai înțelege mai rapid? Aceasta:

```
// Check to see if the employee is eligible for full benefits
if ((employee.flags & HOURLY_FLAG) &&
    (employee.age > 65))
```

Sau aceasta?

```
if (employee.isEligibleForFullBenefits())
```

Este nevoie de doar câteva secunde de gândire pentru a explica cea mai mare parte a intenției voastre în cod. În multe cazuri, este pur și simplu o problemă de a crea o funcție care spune același lucru ca și comentariul pe care doriți să-l scrieți.

Comentarii bune

Unele comentarii sunt necesare sau benefice. Rețineți că singurul comentariu cu adevărat bun este comentariul pe care l-ați găsit o modalitate de a nu-l scrie.

Comentarii proaste

- Majoritatea comentariilor se încadrează în această categorie. De obicei, acestea sunt scuze pentru cod slab sau justificări pentru decizii insuficiente. O **regulă** este nu folosi un comentariu când poți utiliza o funcție sau o variabilă.
- Marcatori de poziție:** uneori, programatorilor le place să marcheze o poziție particulară într-un fișier sursă.

▼ Exemplu

```
// Actions //////////////////////////////////////
```

- Unii programatori pun, la începutul sursei, informații despre autor, data ultimei modificări a codului și alte informații.

4. Formatare

Formatare verticală

1. Cât de mare ar trebui să fie un fișier sursă?
2. Deschidere verticală între concepte
3. Aproape tot codul este citit de la stânga la dreapta și de sus în jos.
4. Fiecare linie reprezintă o expresie sau o clauză, iar fiecare grup de linii reprezintă o gândire completă.
5. Aceste gânduri ar trebui separate între ele cu linii goale.<
6. Conceptele de distanță verticală care sunt strâns legate trebuie să fie păstrate vertical unul de altul.

Declarații variabile

1. Variabilele trebuie declarate cât mai aproape de utilizarea lor.
2. Deoarece funcțiile noastre sunt foarte scurte, variabilele locale ar trebui să apară un top al fiecărei funcții.
3. Funcții dependente. Dacă o funcție apelează alta, acestea trebuie să fie verticale, iar apelantul ar trebui să se afle deasupra străzii, dacă este posibil. Acest lucru oferă programului un flux natural.

Indentare

1. Pentru a face vizibilă ierarhia de scopuri, indentăm liniile de cod sursă proporțional cu poziția lor în ierarhie.
2. Declarațiile la nivelul dosarului nu sunt deloc indentificate.
3. Metodele din cadrul unei clase sunt indentate cu un nivel la dreapta clasei.
4. Implementarea acestor metode este implementată la un nivel la dreapta declarației de metodă.
5. Implementările blocului sunt puse în aplicare la un nivel la dreapta blocului lor conținător, etc.

Testarea codului

1. Prima lege: Nu este posibil să scrieți cod până când nu ați scris un test de unitate defect.
2. A doua lege: Este posibil să nu scrieți mai mult de un test de unitate decât este suficient pentru a eșua și nu a face un eșec.
3. A treia lege: Este posibil să nu scrieți mai multe coduri de producție decât este suficient pentru a trece testul care eșuează.

Bibliografie

The Clean Coder: A Code of Conduct for Professional Programmers (Robert C. Martin Series) Clean Code: A Handbook of Agile Software Craftsmanship (Robert C. Martin Series)